

Práctica gRPC

Mauricio Joel Flores Garzofino

Parte 1:

Servidor:

```
~/C/C0/MicroserviciosFGMJ/Pr/Practica-5/grpc-estudiantes P master node server.js
Servidor gRPC escuchando en 50051
(node:8508) DeprecationWarning: Calling start() is no longer necessary. It can be safely omitted.
(Use `node --trace-deprecation ...` to show where the warning was created)
```

Cliente:

```
~/C/C0/MicroserviciosFGMJ/Pr/Practica-5/grpc-estudiantes master node client.js

Estudiante agregado: {
  ci: '12345',
  nombres: 'Carlos',
  apellidos: 'Montellano',
  carrera: 'Sistemas'
}
Estudiante obtenido: {
  ci: '12345',
  nombres: 'Carlos',
  apellidos: 'Montellano',
  carrera: 'Sistemas'
}
Lista de estudiantes: [
  {
    ci: '12345',
    nombres: 'Carlos',
    apellidos: 'Montellano',
    carrera: 'Sistemas'
  }
]
```

Parte 2: (Capturas de la consola al final)

1. Preparación del entorno

mkdir cursos-estudiantes

cd cursos-estudiantes

npm init -y

Instala dependencias:

npm install @grpc/grpc-js @grpc/proto-loader

2. Contrato .proto

Crea un carpeta proto y dentro universidad.proto:

syntax = "proto3";

package universidad;

import "google/protobuf/empty.proto";

// ===== ENTIDADES =====

message Estudiante {

string ci = 1;

string nombres = 2;

string apellidos = 3;

```

    string carrera = 4;
}

message Curso {
    string codigo = 1;
    string nombre = 2;
    string docente = 3;
}

// ===== REQUESTS / RESPONSES =====
message EstudianteRequest {
    string ci = 1;
}

message CursoRequest {
    string codigo = 1;
}

message EstudianteResponse {
    Estudiante estudiante = 1;
}

message CursoResponse {
    Curso curso = 1;
}

message InscripcionRequest {
    string ci = 1;    // CI del estudiante
    string codigo = 2; // código del curso
}

message InscripcionResponse {
    Estudiante estudiante = 1;
    Curso curso = 2;
}

message EstudiantesList {
    repeated Estudiante estudiantes = 1;
}

message CursosList {
    repeated Curso cursos = 1;
}

```

```
// ===== SERVICIO =====
service UniversidadService {
  rpc AgregarEstudiante (Estudiante) returns (EstudianteResponse);
  rpc AgregarCurso (Curso) returns (CursoResponse);

  rpc InscribirEstudiante (InscripcionRequest) returns (InscripcionResponse);

  rpc ListarCursosDeEstudiante (EstudianteRequest) returns (CursosList);
  rpc ListarEstudiantesDeCurso (CursoRequest) returns (EstudiantesList);

  // opcionalmente útil para debug:
  rpc ListarEstudiantes (google.protobuf.Empty) returns (EstudiantesList);
  rpc ListarCursos (google.protobuf.Empty) returns (CursosList);
}
```

3. Implementar el servidor

Crear un archivo [server.js](#)

```
import grpc from "@grpc/grpc-js";
import protoLoader from "@grpc/proto-loader";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const PROTO_PATH = path.resolve(__dirname, "proto", "universidad.proto");

// Cargar el proto (opciones útiles para defaults y enums/longs como strings)
const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  includeDirs: [path.resolve(__dirname, "proto")],
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true
});

const proto = grpc.loadPackageDefinition(packageDefinition).universidad;

// ===== Base de datos en memoria =====
const estudiantes = []; // Estudiante[]
const cursos = []; // Curso[]

// Relación muchos-a-muchos:
// - Map de ci -> Set de codigos de curso
```

```

// - Map de codigo -> Set de cis (estudiantes)
const cursosPorEstudiante = new Map(); // Map<string, Set<string>>
const estudiantesPorCurso = new Map(); // Map<string, Set<string>>

// ===== Helpers =====
function buscarEstudiante(ci) {
  return estudiantes.find(e => e.ci === ci) || null;
}
function buscarCurso(codigo) {
  return cursos.find(c => c.codigo === codigo) || null;
}

function inscribir(ci, codigo) {
  if (!cursosPorEstudiante.has(ci)) cursosPorEstudiante.set(ci, new Set());
  if (!estudiantesPorCurso.has(codigo)) estudiantesPorCurso.set(codigo, new Set());
  const setCursos = cursosPorEstudiante.get(ci);
  const setEstudiantes = estudiantesPorCurso.get(codigo);

  if (setCursos.has(codigo)) {
    // ya inscrito
    const err = {
      code: grpc.status.ALREADY_EXISTS,
      message: `El estudiante ${ci} ya está inscrito en el curso ${codigo}`
    };
    throw err;
  }

  setCursos.add(codigo);
  setEstudiantes.add(ci);
}

// ===== Implementación de servicios =====
const serviceImpl = {
  AgregarEstudiante: (call, callback) => {
    const nuevo = call.request; // Estudiante
    if (!nuevo.ci || !nuevo.nombres) {
      return callback({
        code: grpc.status.INVALID_ARGUMENT,
        message: "ci y nombres son obligatorios"
      });
    }
    const yaExiste = buscarEstudiante(nuevo.ci);
    if (yaExiste) {
      return callback({

```

```

        code: grpc.status.ALREADY_EXISTS,
        message: `Ya existe un estudiante con ci ${nuevo.ci}`
    });
}
estudiantes.push(nuevo);
callback(null, { estudiante: nuevo });
},

```

```

AgregarCurso: (call, callback) => {
    const nuevo = call.request; // Curso
    if (!nuevo.codigo || !nuevo.nombre) {
        return callback({
            code: grpc.status.INVALID_ARGUMENT,
            message: "codigo y nombre son obligatorios"
        });
    }
    const yaExiste = buscarCurso(nuevo.codigo);
    if (yaExiste) {
        return callback({
            code: grpc.status.ALREADY_EXISTS,
            message: `Ya existe un curso con código ${nuevo.codigo}`
        });
    }
    cursos.push(nuevo);
    callback(null, { curso: nuevo });
},

```

```

InscribirEstudiante: (call, callback) => {
    const { ci, codigo } = call.request;
    const est = buscarEstudiante(ci);
    const cur = buscarCurso(codigo);

    if (!est) {
        return callback({ code: grpc.status.NOT_FOUND, message: `Estudiante ${ci} no encontrado` });
    }
    if (!cur) {
        return callback({ code: grpc.status.NOT_FOUND, message: `Curso ${codigo} no encontrado` });
    }
    try {
        inscribir(ci, codigo);
        callback(null, { estudiante: est, curso: cur });
    } catch (err) {

```

```

        callback(err);
    }
},

ListarCursosDeEstudiante: (call, callback) => {
    const { ci } = call.request;
    const est = buscarEstudiante(ci);
    if (!est) {
        return callback({ code: grpc.status.NOT_FOUND, message: `Estudiante ${ci} no
encontrado` });
    }
    const codigos = Array.from(cursosPorEstudiante.get(ci) || []);
    const cursosDelEst = cursos.filter(c => codigos.includes(c.codigo));
    callback(null, { cursos: cursosDelEst });
},

ListarEstudiantesDeCurso: (call, callback) => {
    const { codigo } = call.request;
    const cur = buscarCurso(codigo);
    if (!cur) {
        return callback({ code: grpc.status.NOT_FOUND, message: `Curso ${codigo} no
encontrado` });
    }
    const cis = Array.from(estudiantesPorCurso.get(codigo) || []);
    const ests = estudiantes.filter(e => cis.includes(e.ci));
    callback(null, { estudiantes: ests });
},

// RPCs opcionales (útiles para verificar estado)
ListarEstudiantes: (_call, callback) => {
    callback(null, { estudiantes });
},
ListarCursos: (_call, callback) => {
    callback(null, { cursos });
}
};

// ===== Boot del servidor =====
const server = new grpc.Server();
server.addService(proto.UniversidadService.service, serviceImpl);

const PORT = "50051";
server.bindAsync(`0.0.0.0:${PORT}`, grpc.ServerCredentials.createInsecure(), (err, bindPort) =>
{

```

```

    if (err) {
      console.error("Error al iniciar servidor:", err);
      return;
    }
    console.log(`Servidor gRPC escuchando en ${bindPort}`);
    server.start();
  });

```

4. Implementar el cliente

```

import grpc from "@grpc/grpc-js";
import protoLoader from "@grpc/proto-loader";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const PROTO_PATH = path.resolve(__dirname, "proto", "universidad.proto");
const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  includeDirs: [path.resolve(__dirname, "proto")],
});
const proto = grpc.loadPackageDefinition(packageDefinition).universidad;

const client = new proto.UniversidadService(
  "localhost:50051",
  grpc.credentials.createInsecure()
);

function agregarEstudiante(est) {
  return new Promise((resolve, reject) => {
    client.AgregarEstudiante(est, (err, res) => (err ? reject(err) : resolve(res.estudiante)));
  });
}

function agregarCurso(cur) {
  return new Promise((resolve, reject) => {
    client.AgregarCurso(cur, (err, res) => (err ? reject(err) : resolve(res.curso)));
  });
}

function inscribir(ci, codigo) {
  return new Promise((resolve, reject) => {
    client.InscribirEstudiante({ ci, codigo }, (err, res) => (err ? reject(err) : resolve(res)));
  });
}

```

```
}
```

```
function listarCursosDeEstudiante(ci) {  
  return new Promise((resolve, reject) => {  
    client.ListarCursosDeEstudiante({ ci }, (err, res) => (err ? reject(err) : resolve(res.cursos)));  
  });  
}
```

```
function listarEstudiantesDeCurso(codigo) {  
  return new Promise((resolve, reject) => {  
    client.ListarEstudiantesDeCurso({ codigo }, (err, res) => (err ? reject(err) :  
    resolve(res.estudiantes)));  
  });  
}
```

```
(async () => {  
  try {  
    // Registro de estudiante  
    const estudiante = await agregarEstudiante({  
      ci: "12345",  
      nombres: "Mauricio",  
      apellidos: "Montellano",  
      carrera: "Sistemas",  
    });  
    console.log("Estudiante agregado:", estudiante);  
  
    // Registro de cursos  
    const curso1 = await agregarCurso({ codigo: "COM600", nombre: "Microservicios",  
    docente: "Ing. Montellano" });  
    const curso2 = await agregarCurso({ codigo: "SIS256", nombre: "WEB", docente: "Ing.  
    Montellano" });  
    console.log("Cursos agregados:", curso1, curso2);  
  
    // Inscripción estudiante - cursos  
    await inscribir("12345", "COM600");  
    await inscribir("12345", "SIS256");  
    console.log("Inscripciones realizadas.");  
  
    // Intentar inscribir duplicado para ver ALREADY_EXISTS  
    try {  
      await inscribir("12345", "COM600");  
    } catch (e) {  
      console.log("Inscripción duplicada:", e.code === 6 ? "ALREADY_EXISTS" : e.message);  
    }  
  }  
})
```



```

//Consultar los cursos del estudiante
const cursosDelEst = await listarCursosDeEstudiante("12345");
console.log("Cursos del estudiante 12345:", cursosDelEst);

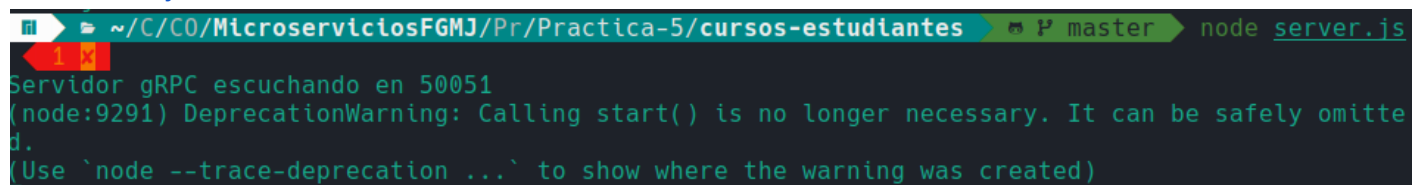
// Consultar los estudiantes de un curso
const estudiantesDeCOM600 = await listarEstudiantesDeCurso("COM600");
console.log("Estudiantes en COM600:", estudiantesDeCOM600);
} catch (err) {
  console.error("Error en cliente:", err);
}
})();

```

5. Ejecución

Terminal 1: Ejecutar el servidor

node [server.js](#)



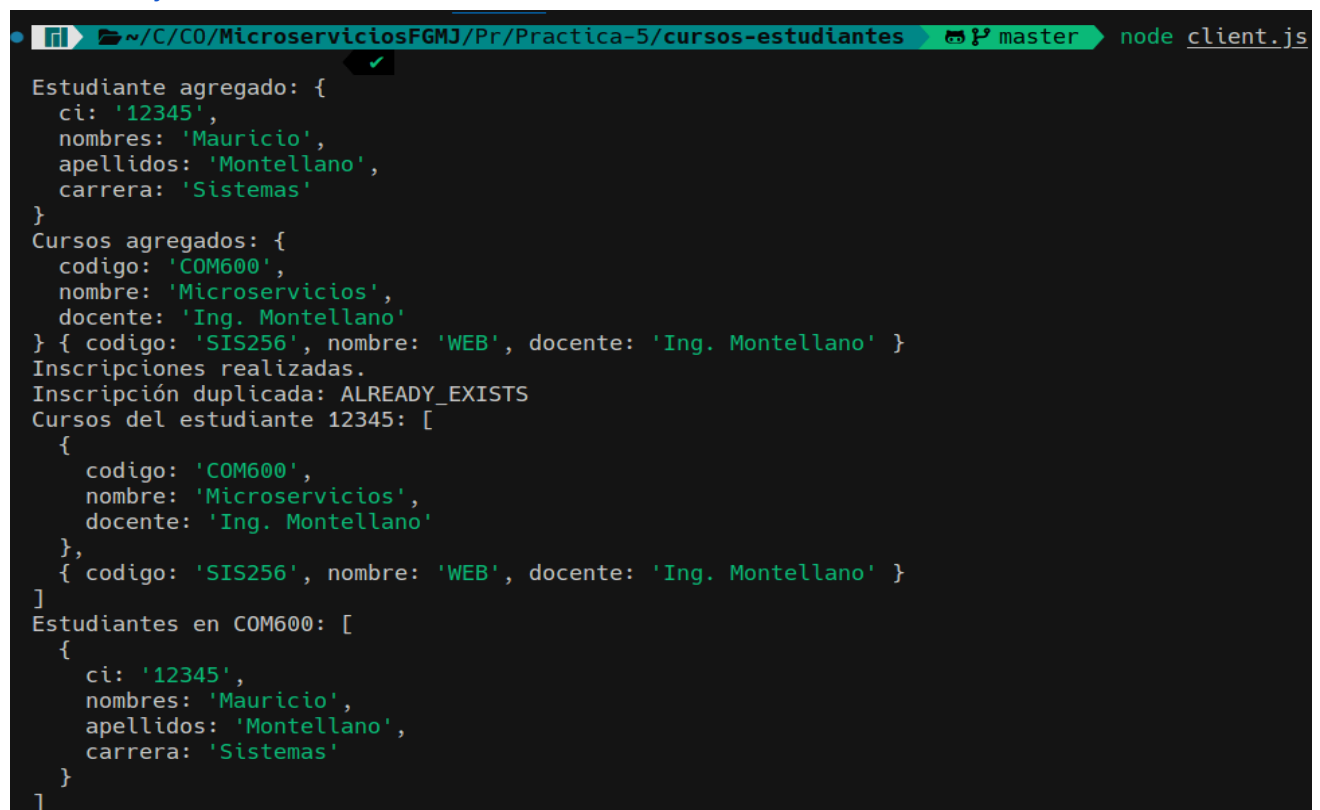
```

~/C/C0/MicroserviciosFGMJ/Pr/Practica-5/cursos-estudiantes master node server.js
Servidor gRPC escuchando en 50051
(node:9291) DeprecationWarning: Calling start() is no longer necessary. It can be safely omitted.
(Use `node --trace-deprecation ...` to show where the warning was created)

```

Terminal 2: Ejecutar Cliente

node [client.js](#)



```

~/C/C0/MicroserviciosFGMJ/Pr/Practica-5/cursos-estudiantes master node client.js
Estudiante agregado: {
  ci: '12345',
  nombres: 'Mauricio',
  apellidos: 'Montellano',
  carrera: 'Sistemas'
}
Cursos agregados: {
  codigo: 'COM600',
  nombre: 'Microservicios',
  docente: 'Ing. Montellano'
} { codigo: 'SIS256', nombre: 'WEB', docente: 'Ing. Montellano' }
Inscripciones realizadas.
Inscripción duplicada: ALREADY_EXISTS
Cursos del estudiante 12345: [
  {
    codigo: 'COM600',
    nombre: 'Microservicios',
    docente: 'Ing. Montellano'
  },
  { codigo: 'SIS256', nombre: 'WEB', docente: 'Ing. Montellano' }
]
Estudiantes en COM600: [
  {
    ci: '12345',
    nombres: 'Mauricio',
    apellidos: 'Montellano',
    carrera: 'Sistemas'
  }
]

```